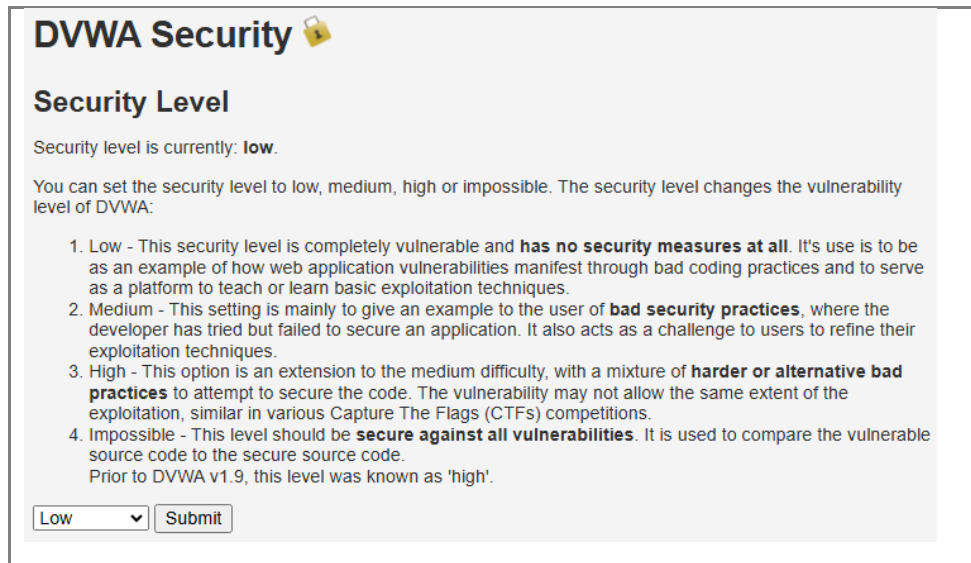


**AIM:** To demonstrate a basic SQL Injection attack and a simple session hijacking attack using cookie manipulation against the DVWA application running on the local server.

### PROCEDURE:

1. Start XAMPP (Apache and MySQL) and login to <http://localhost/dvwa/> with credentials admin / password. Set DVWA Security to 'Low'.



**DVWA Security** 

### Security Level

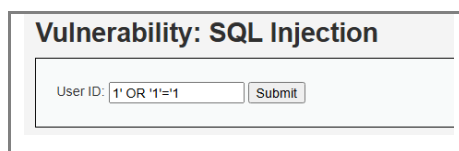
Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.  
Prior to DVWA v1.9, this level was known as 'high'.

Low

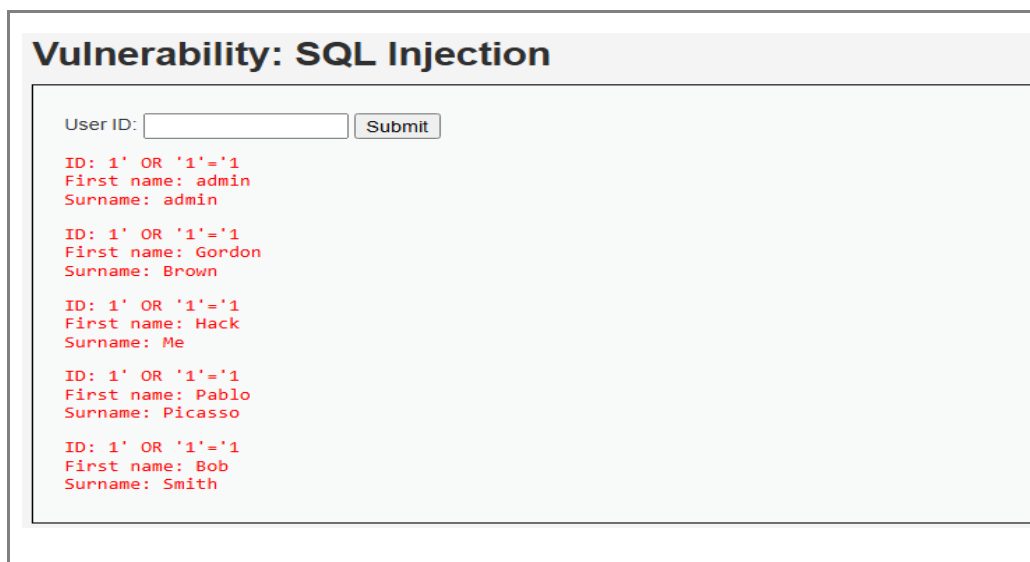
2. Open the SQL Injection module. In the User ID field, enter the payload `1' OR '1'='1` and click Submit.



**Vulnerability: SQL Injection**

User ID:

3. Observe that all user records (id, first name, surname) from the underlying database are returned, proving that user input is concatenated directly into the SQL query.



**Vulnerability: SQL Injection**

User ID:

```
ID: 1' OR '1'='1
First name: admin
Surname: admin

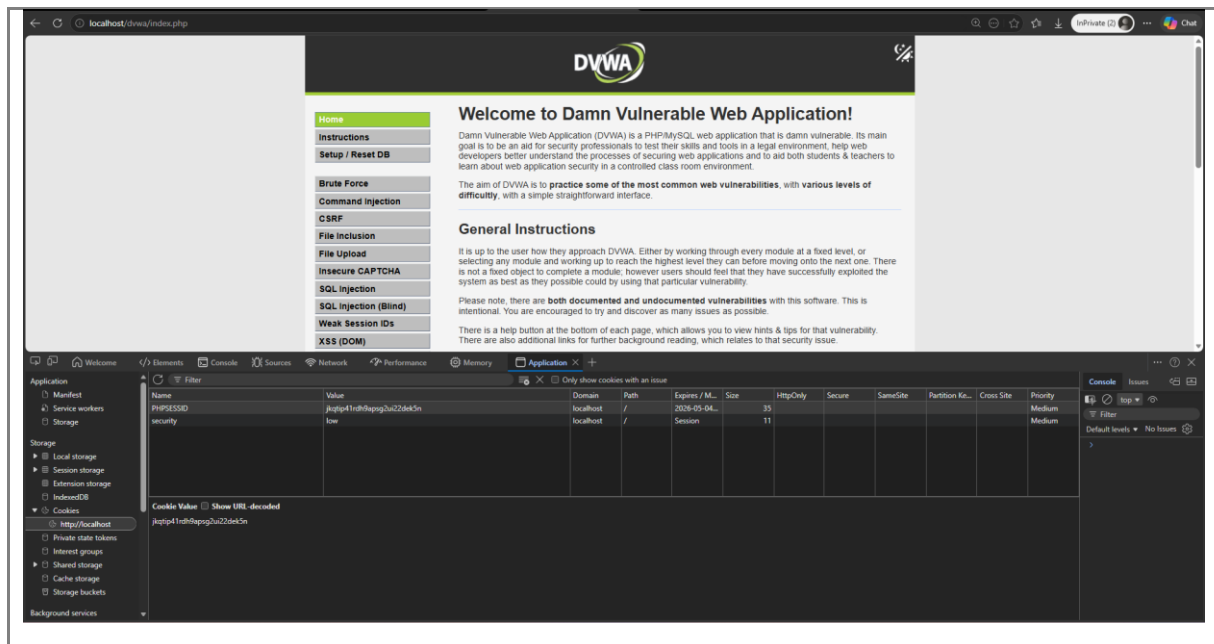
ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

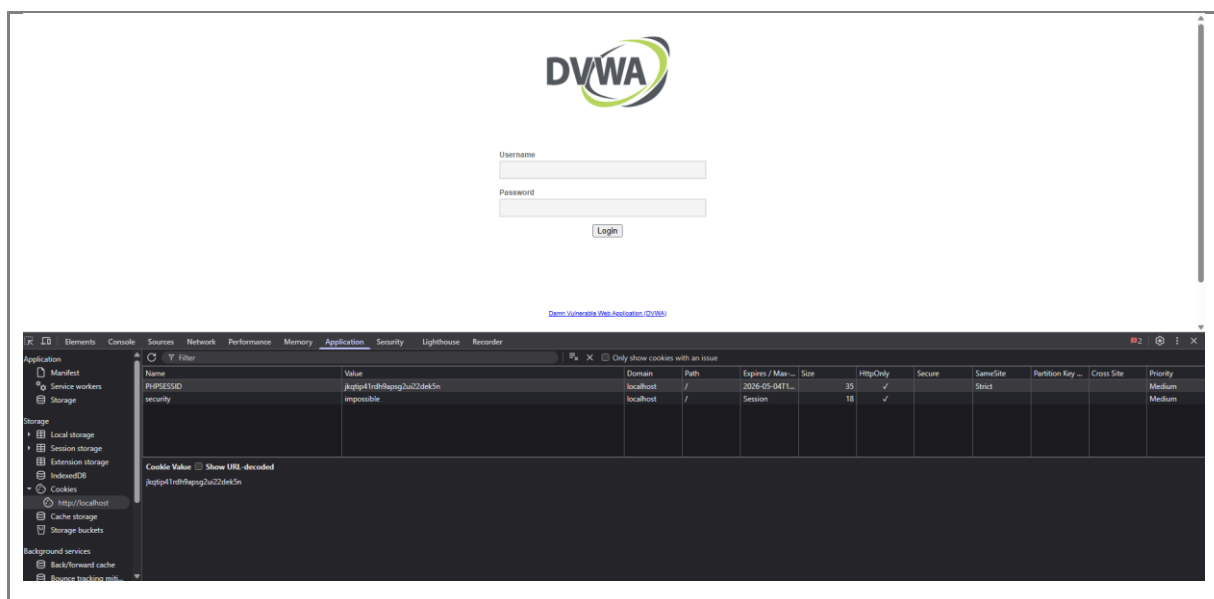
ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith
```

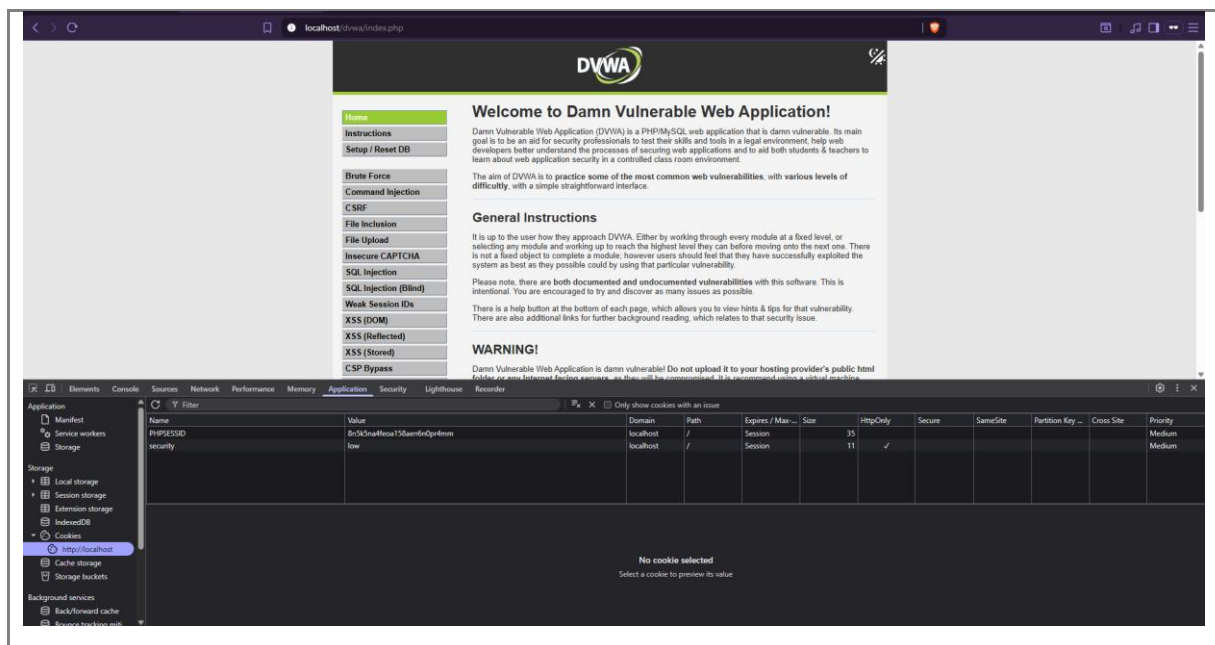
4. Now demonstrate session hijacking. In one browser, login as admin and open Developer Tools (F12) > Application > Cookies and copy the value of the PHPSESSID cookie.



5. Open a different browser (or incognito window) where you are not logged in. Open DevTools > Application > Cookies and create a new PHPSESSID cookie with the value copied above. Refresh the page.



6. Observe that the second browser is now logged in as admin without ever entering a password. This demonstrates that stealing a session cookie is enough to hijack a session.



**RESULT:** Thus, a basic SQL Injection attack and a cookie-based session hijacking attack were performed successfully on DVWA.